Aufgabe 1

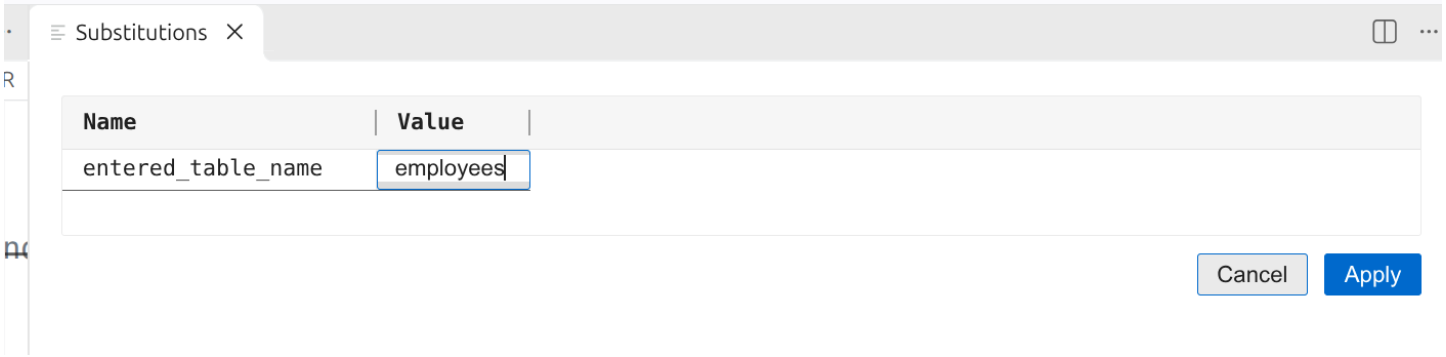
1a)

```
--a)
SELECT *
FROM dictionary
WHERE table_name LIKE 'USER_T%';

-- USER_TAB_COLUMNS sieht gut aus: Columns of user's tables, views and clusters
-- mal reinschauen

SELECT * FROM USER_TAB_COLUMNS;
-- sieht gut aus! alle gewollten Infos stehen darin.


SELECT column_name,
    data_type,
    data_type,
    data_length,
    nullable,
    data_precision AS ct_digits,
    data_scale AS ct_decimal_digits
FROM USER_TAB_COLUMNS
WHERE table_name = UPPER('&entered_table_name');
```



Ausgabe bei Eingabe von employees

	COLUMN_NAME	DATA_TYPE	DATA_TYPE	DATA_LENGTH	NULLABLE	CT_DIGITS	CT_DECIMAL_DIGITS
1	EMPLOYEE_ID	NUMBER	NUMBER	22	N	6	0
2	FIRST_NAME	VARCHAR2	VARCHAR2	20	Y	(null)	(null)
3	LAST_NAME	VARCHAR2	VARCHAR2	25	N	(null)	(null)
4	EMAIL	VARCHAR2	VARCHAR2	25	N	(null)	(null)
5	PHONE_NUMBER	VARCHAR2	VARCHAR2	20	Y	(null)	(null)
6	HIRE_DATE	DATE	DATE	7	N	(null)	(null)
7	JOB_ID	VARCHAR2	VARCHAR2	10	N	(null)	(null)
8	SALARY	NUMBER	NUMBER	22	Y	8	2
9	COMMISSION_PCT	NUMBER	NUMBER	22	Y	2	2
10	MANAGER_ID	NUMBER	NUMBER	22	Y	6	0
11	DEPARTMENT_ID	NUMBER	NUMBER	22	Y	4	0

1b)

Code

```
SELECT * FROM USER_TAB_COLUMNS WHERE table_name = 'EMPLOYEES';
SELECT * FROM USER_IND_columns WHERE table_name = 'FILM_ACTOR';
SELECT * FROM USER_IND_columns WHERE index_name = 'FILM_ACTOR_PK';


SELECT index_name,
LISTAGG(column_name, ', ')
WITHIN GROUP (ORDER BY column_position) AS columns
FROM USER_IND_COLUMNS
WHERE index_name LIKE '%FILM%'
GROUP BY index_name;
```

Ausgabe

	INDEX_NAME	COLUMNS
1	FILM_ACTOR_PK	ACTOR_ID, FILM_ID
2	FILM_CATEGORY_PK	FILM_ID, CATEGORY_ID
3	FILM_PK	FILM_ID

3 selected

1c)

Code

```
WITH tmp_table AS (
SELECT segment_name AS name,
       bytes AS size_bytes,
       DENSE_RANK() OVER (
            ORDER BY bytes DESC
       ) AS dense_rank
FROM user_segments
WHERE segment_type = 'TABLE'
)

SELECT name, size_bytes
FROM tmp_table
WHERE dense_rank <= 3;
```

Ausgabe

	NAME	SIZE_BYTES
1	RENTAL	917504
2	PAYMENT	851968
3	FILM	262144

3 selected

1d)

Code

```
SELECT * FROM USER_OBJECTS;

SELECT DISTINCT object_type
FROM user_objects
ORDER BY object_type;

SELECT CASE WHEN object_type = 'TABLE' THEN
              'DROP TABLE' || ' ' || object_name || ' CASCADE CONSTRAINTS' || ';'
            ELSE
              'DROP ' || object_type || ' ' || object_name || ';'
            END AS drop_statement
FROM user_objects
WHERE object_type IN (
      'SEQUENCE',
      'TABLE',
      'VIEW',
      'MATERIALIZED VIEW'
);
```

Ausgabe

	DROP_STATEMENT
1	DROP TABLE ACTOR CASCADE CONSTRAINTS;
2	DROP VIEW ACTOR_AGE;
3	DROP TABLE ACTOR_AGE_DAILY CASCADE CONSTRAINTS;
4	DROP MATERIALIZED VIEW ACTOR_AGE_DAILY;
...	
33	DROP TABLE STORE CASCADE CONSTRAINTS;

33 selected

Aufgabe 2

2a

Code

```
--a alter der Schauspieler zum aktuellen Tag,
-- virtuelle Sicht dazu erstellen und abfragen

CREATE OR REPLACE VIEW actor_age AS
SELECT actor_id, first_name, last_name,
    TRUNC(MONTHS_BETWEEN(SYSDATE, birth_date) / 12) AS age
FROM actor
ORDER BY actor_id;

SELECT *
FROM actor_age;
```

Ausgabe

	ACTOR_ID	FIRST_NAME	LAST_NAME	AGE
1	1	PENELOPE	GUINESS	75
2	2	NICK	WAHLBERG	66
3	3	ED	CHASE	56
4	4	JENNIFER	DAVIS	74
5	5	JOHNNY	LOLLOBRIGIDA	79
6	6	BETTE	NICHOLSON	71

...

200	200	THORA	TEMPLE	65

200 selected

2b

Code

```
CREATE MATERIALIZED VIEW actor_age_daily
    REFRESH COMPLETE
    START WITH TRUNC(SYSDATE + 1) + 1/24
    NEXT TRUNC(SYSDATE + 1) + 1/24
AS
SELECT *
FROM actor_age;

-- abfragen, für die doku
SELECT * FROM actor_age_daily;
```

Ausgabe

```
-- selbe Ausgabe wie in 2a
```

2c

Code

```
--c nur in der Nacht von Sonntag auf montag 3.00 uhr
CREATE MATERIALIZED VIEW actor_age_weekly
    REFRESH COMPLETE
    START WITH TRUNC((SYSDATE + 7), 'IW') + 3/24
    NEXT TRUNC((SYSDATE + 7), 'IW') + 3/24
AS
SELECT *
FROM actor_age;

SELECT * FROM actor_age_weekly;
```

Ausgabe

```
-- selbe Ausgabe wie in 2a
```

2d)

Ich suche den Zeitpunkt der letzten und nächsten Aktualisierung im Data Dictionary.
Also:

```
-- nach dem Stichwort refresh
SELECT *
FROM dictionary
WHERE table_name LIKE '%refresh%'
OR comments LIKE '%refresh%';
```

	TABLE_NAME	COMMENTS
1	USER_REFRESH	All the refresh groups
2	USER_REFRESH_CHILDREN	All the objects in refresh groups, where the user owns the refresh group
3	USER_MVIEW_REFRESH_TIMES	Materialized views and their last refresh times for each master table that the user can look at
4	ALL_MVIEW_REFRESH_TIMES	Materialized views and their last refresh times for each master table that the user can look at
5	ALL_REFRESH	All the refresh groups that the user can touch
6	ALL_REFRESH_CHILDREN	All the objects in refresh groups, where the user can touch the group
7	ALL_REFRESH_DEPENDENCIES	Description of the detail tables that materialized views depend on for refresh

Ich suche also weiter in: USER_MVIEW_REFRESH_TIMES

```
SELECT *
FROM USER_MVIEW_REFRESH_TIMES;
```

	OWNER	NAME	MASTER_OWNER	MASTER	LAST_REFRESH
1	S2510307051	ACTOR_AGE_DAILY	S2510307051	ACTOR	10-NOV-26
2	S2510307051	ACTOR_AGE_WEEKLY	S2510307051	ACTOR	10-NOV-26

LAST_REFRESH habe ich also gefunden - heute, der 10. November wurde die Sicht erstellt, daher der letzte refresh auch heute.
Weitersuchen nach dem next refresh

Ich probiere USER_REFRESH:

```
SELECT *
FROM USER_REFRESH;
```

	ROWNER	RNAME	REFGROUP	IMPLICIT_DESTROY	PUSH_DEFERRED_RPC	REFRESH_AFTER_ERRORS	ROLLBACK_SEG	JOB	NEXT_DATE	INTERVAL
1	S2510307051	ACTOR_AGE_DAILY	2753	Y	Y	N	(null)	0	11-NOV-26	TRUNC(SYSDATE + 1) + 1/24
2	S2510307051	ACTOR_AGE_WEEKLY	2756	Y	Y	N	(null)	0	16-NOV-26	TRUNC((SYSDATE + 7), 'IW') + 3/24

Top, hier steht das next_date. Nochmal schöner:

```
SELECT rname, next_date
FROM USER_REFRESH;
```

	RNAME	NEXT_DATE
1	ACTOR_AGE_DAILY	11-NOV-26
2	ACTOR_AGE_WEEKLY	16-NOV-26

Interessanterweise findet man den last_refresh und das next_date in zwei unterschiedlichen Tabellen.
Wir haben aber beide gefunden.

2e)

Code

```
DROP MATERIALIZED VIEW actor_age_weekly;
DROP MATERIALIZED VIEW actor_age_daily;
DROP VIEW actor_age;
```

Ausgabe

```
Materialized view ACTOR_AGE_WEEKLY dropped.


Materialized view ACTOR_AGE_DAILY dropped.


View ACTOR_AGE dropped.
```

Aufgabe 3

3a)

Code

```
-- alle employees die genau 2 levels unter 102 sind
-- level 1 ist 102
-- zwei darunter ist level 3
SELECT employee_id, last_name, manager_id, LEVEL
FROM employees
WHERE LEVEL = 3
START WITH employee_id = 102
CONNECT BY PRIOR employee_id = manager_id
      AND LEVEL <= 3; -- Rekursion begrenzen
-- heißt: geh weiter wo die manager_id 102 ist (beim ersten Schritt)
```

Ausgabe

	EMPLOYEE_ID	LAST_NAME	MANAGER_ID	LEVEL
1	104	Ernst	103	3
2	107	Lorentz	103	3

2 selected

3b)

Code

```
SELECT employee_id, last_name
FROM employees
WHERE last_name = 'King';
-- King -- 100

SELECT employee_id, last_name, manager_id, department_id
FROM employees
WHERE department_id != PRIOR department_id
START WITH employee_id = 100
CONNECT BY NOCYCLE PRIOR employee_id = manager_id;
```

Ausgabe

	EMPLOYEE_ID	LAST_NAME	MANAGER_ID	DEPARTMENT_ID
1	200	Whalen	101	10
2	205	Higgins	101	110
3	103	Hunold	102	60
4	124	Mourgos	100	50
5	149	Zlotkey	100	80
6	201	Hartstein	100	20

6 selected

3c)

Code

```
WITH tmp_table AS (
      SELECT CONNECT_BY_ROOT employee_id AS root_manager_id,
            -- jede id wird rek. durchlaufen
            -- erste id ist also manager aller anderen
            CONNECT_BY_ROOT last_name AS root_manager_name,
            employee_id, last_name,
            manager_id, hire_date
      FROM employees
      WHERE hire_date < CONNECT_BY_ROOT hire_date
      -- hire_date von Untergebenem war vorher
      CONNECT BY NOCYCLE PRIOR employee_id = manager_id
)

SELECT root_manager_id, root_manager_name,
      COUNT(*) AS untergebene
FROM tmp_table
GROUP BY root_manager_id, root_manager_name;
```

Ausgabe

	ROOT_MANAGER_ID	ROOT_MANAGER_NAME	UNTERGEBENE
1	101	Kochhar	1
2	124	Mourgos	4
3	149	Zlotkey	3
4	102	De Haan	2

4 selected

Aufgabe 4

4a)

Code

```
CREATE OR REPLACE VIEW partners AS
SELECT film.film_id AS film_id,
       id1.actor_id AS actor_id1,
       id2.actor_id AS actor_id2
FROM film_actor id1
       JOIN film_actor id2
              ON id1.film_id = id2.film_id
       JOIN film
              ON id1.film_id = film.film_id
WHERE release_year = 1990
       AND id1.actor_id <> id2.actor_id;

SELECT *
FROM partners;
```

Ausgabe

	FILM_ID	ACTOR_ID1	ACTOR_ID2
1	597	11	28
2	597	13	28
3	597	64	28
4	597	129	28
5	597	184	28
...			
921	714	166	182
922	714	200	182

922 selected

4b)

Code

```
-- actor_ids der beiden:
-- Judy Dean: 35
SELECT actor_id, last_name, first_name
FROM actor
WHERE last_name = 'DEAN' AND first_name = 'JUDY';

-- Russell Close: 183
SELECT actor_id, last_name, first_name
FROM actor
WHERE last_name = 'CLOSE' AND first_name = 'RUSSELL';

-- von den beiden die view durchlaufen,
-- nur bis level 3

WITH partners_reached AS (
   SELECT DISTINCT actor_id2
   FROM partners
   START WITH actor_id1 IN (183, 35)
   CONNECT BY NOCYCLE PRIOR actor_id2 = actor_id1
   AND LEVEL <= 3
   AND PRIOR film_id <> film_id
)
SELECT pre.actor_id2, act.first_name, act.last_name
FROM partners_reached pre
JOIN actor act ON pre.actor_id2 = act.actor_id;
```

Ausgabe

	ACTOR_ID2	FIRST_NAME	LAST_NAME
1	194	MERYL	ALLEN
2	3	ED	CHASE
3	10	CHRISTIAN	GABLE
4	15	CUBA	OLIVIER
5	20	LUCILLE	TRACY
6	21	KIRSTEN	PALTROW
7	31	SISSY	SOBIESKI

...

48	186	JULIA	ZELLWEGER

48 selected